

SPLATROOM · EXPLORATION ARCHIVE

SplatRoom 合并工具 探索历程图文存档

从功能起点到标记对齐攻坚：全部主线探索与支线探索的完整记录

项目：SplatRoom（基于 PlayCanvas SuperSplat v2.31.10 二次开发）

范围：合并工作台（?mode=merge）全部功能演进 + 关联支线

时段：2026-07 至 2026-08-11（归档日）

作者：摄影师黄Sir（1924705842@qq.com）

归档日期：2026-08-11

目录

01 项目背景与架构

02 探索时间线总览

03 主线：合并工具功能演进

3.1 多模型加载 / 3.2 选择与变换 / 3.3 视角控制 / 3.4 对齐三模式 / 3.5 合并导出

04 核心攻坚：标记对齐（对应点）专项

4.1 功能重构 / 4.2 九轮 Bug 修复全景 / 4.3 最终方案定稿

05 支线探索

5.1 PiP 近小远大 / 5.2 正交视角模型消失 / 5.3 表面平整 L1/L2 / 5.4 平面修整 / 5.5 对比工具 / 5.6 打包发布 / 5.7 无头浏览器复现

06 关键技术结论

07 构建与部署实战经验

08 历次部署链接清单

本文档为工作记忆（.workbuddy/memory）与工作日志的结构化整理，附图均来自开发过程中的真实截图。

1 · 项目背景与架构

SplatRoom 是基于 **PlayCanvas SuperSplat v2.31.10** 二次开发的高斯泼溅（3D Gaussian Splatting）编辑工具，作者为摄影师黄Sir。项目最初位于 `C:\Users\Byon Huang\WorkBuddy\2026-07-20-11-07-50`，后迁移至独立工作空间 `C:\Users\Byon Huang\WorkBuddy\SplatRoom`。

其中「合并工具」（merge）以独立窗口模式运行：访问部署链接加 `?mode=merge` 进入合并工作台，创建**独立的 PlayCanvas app**，渲染链路（canvas / device / app / scene / camera）与主视窗、PiP 完全隔离。

架构决策要点

- 合并场景不 import 主编辑器任何模块，独立自包含。
- 最多同时加载 **8** 个高斯模型（PLY）；原始 PLY 只读，参数通过 `.sscsg` 侧车保存，fragment shader 实时应用、导出时烘焙。
- 多模型通过 `group-renderer.ts` 跨 splat 合并 DataTable → 单一 Entity 渲染；Ctrl+Click 多选 → 链锁。
- 生产构建：rollup → `dist/index.js`（约 6.8MB）；打包用 electron-builder portable 目标（单文件 .exe）。

2 · 探索时间线总览

07 下旬	SplatRoom 初始化、SuperSplat 二次开发基线搭建；SSCG 调色系统 v4（HSL 8 色区 × 3 属性）。
07-24	多模型 Group/Link 支持； <code>group-renderer.ts</code> 跨 splat 合并渲染。
08 初	SOG WebP 纹理修复（MAX_STRIPE_BYTES 8MB→128MB）；运镜系统（惯性动量 / Numpad 预设视角 / 自动旋转 / 双轴面板）；视频导出（setPoseOverride 首帧 keyFrame）。
08-02	平面修整（Planar Fix，后暂停存档）；对比工具（?mode=compare）启动：PCA+ICP 对齐、左右分屏、24 属性直方图、破洞检测（TSPE-GS 透射率）。
08-03	表面平整 L2（surface-refiner.ts）；SSCG 持续推进。
08-04	X/Y/Z 正交正视视角模型消失 bug 定位与修复（gsplatCorner NaN 兜底）；表面平整 L1 多轮改造与整体回退（见 5.3）；确立 headless 复现 WebGL bug 工作流（puppeteer-core + Edge + SwiftShader）。
08-08 ~ 08-10	合并工具主体功能搭建：多模型加载、选择、变换 gizmo、视角控制、自动对齐、合并导出 PLY。
08-11	标记对齐专项攻坚日 ：功能重构 + 九轮 Bug 修复（详见第 4 章）；导出「存储为」对话框及手势修复。本文档归档。

3 · 主线：合并工具功能演进

3.1 多模型加载

文件选择 (`#fileInput`)、URL 输入 (`#fileUrl`)、拖拽 `.ply` 到画面 (橙色虚线遮罩) 三种入口；引擎一次性 boot, `loadModelFromBuffer(buf, name, sizeKB)` 支持多模型切换；`teardownModel()` 负责销毁旧 entity / asset / 选择状态。

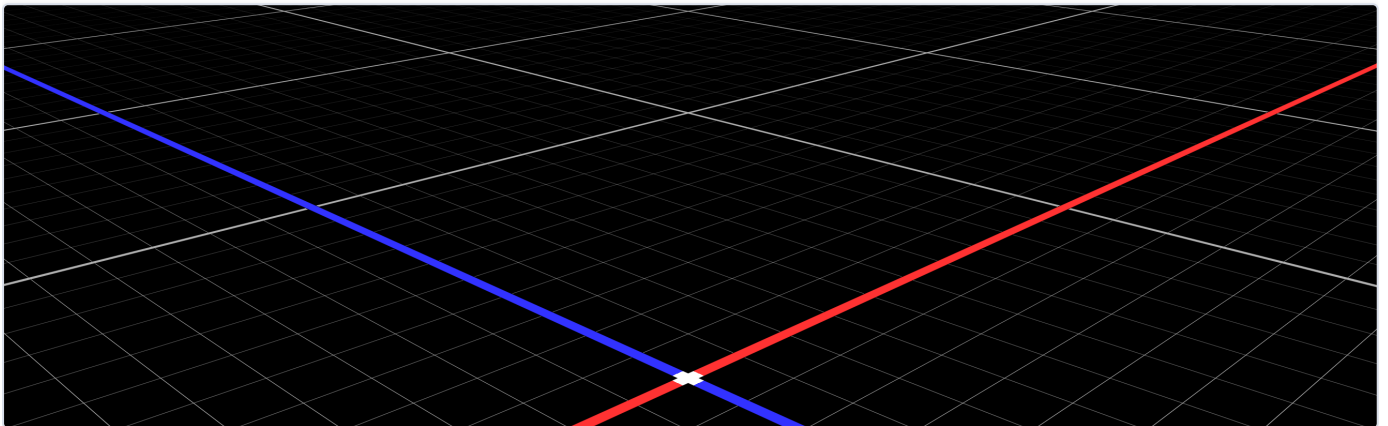


图 3-1 合并工作台界面：多模型并排、选择框、右侧变换/导出面板。

3.2 选择与变换

- 左键单击选中 (Ctrl 多选/反选)；双击聚焦；Ctrl+空格拖拽框选。
- 选中后默认进入「移动」模式，显示 3 轴 gizmo (移动箭头 / 轴端缩放块 / 旋转环)；拖拽箭头移动模型，未命中手柄时拖拽为 orbit。
- 变换面板：左上角浮层，0.01 步进拖拽精确调整 pos/rot/scale。

3.3 视角控制

- 左键拖拽 orbit、中键 pan、右键 look、滚轮缩放。
- 右上角 X/Y/Z 轴导航 overlay (Blender 风格环 + 字母标签，命中测试直接算 6 个轴端屏幕位置取最近)。
- 视窗底部 3 个视角按钮 (俯视 / 正视 / 侧视) → `snapToAxis` 进入正交视角。

俯视退化 bug (08-11 上午，后续为标记对齐让路)

俯视/仰视 ($\text{elevation} \approx \pm 90^\circ$) 时视线与默认 `up=(0,1,0)` 平行，`lookAt` 退化出 NaN → 模型消失。先以 `safeUp(forward)` 硬阈值修复 (视线近竖直改用 Z 轴为 up)，后因左键拖拽轴导航突然翻转 (up 在阈值附近跳变)，改为 `calcCameraUp(forward, azimuth)` 平滑 blend；轴 overlay 与 hit test 改用 `camera.getWorldTransform().getX()/getY()` 世界基，不再手动 recompute。

3.4 对齐三模式

模式	实现	适用场景
手动对齐 (manual)	拖拽 gizmo 手工摆放	粗调
自动对齐 (auto)	PCA 预对齐 + Kabsch / 符号对齐 / 变换对齐 (<code>merge-align.ts</code>)	整体刚体对齐
标记对齐 (marker)	用户在两模型上点 3 组对应点 → Kabsch 求刚体变换, 模型 2 对齐到模型 1	精细特征对齐

3.5 合并导出

- `exportMergedPly()` : 合并多模型 `gsplatData` → 单一 PLY (高斯位置 / 旋转 / SH 颜色 / 不透明度) 。
- 存储为对话框**: 优先 `window.showSaveFilePicker` (Chromium 桌面版 https 安全上下文), 用户可自定文件名与位置; 取消抛 `AbortError`; 不支持抛 `NOT_SUPPORTED`, 回退到已选文件夹或直接下载。

导出失败修复 (08-11 13:44)

报错 `Must be handling a user gesture to show a file picker` 。**根因**: **handler 先 `await exportMerged()` 再调 `picker`, `await` 之后 `transient activation` 丢失** 。**修复**: `saveWithPicker(name, getData)` 把数据改为回调——函数内先同步拿 `handle`, 再调回调取数据。

```
// 正确: picker 必须在用户手势同步链上
const saved = await saveWithPicker(name, async () => {
  const r = await scene.exportMerged(name);
  return r.data;
});
```



图 3-2 导出失败报错: `showSaveFilePicker` 需要用户手势。

4 · 核心攻坚：标记对齐（对应点）专项

标记对齐是 08-11 当天的主战场：从功能重构到连续九轮 Bug 修复，最终沉淀出「射线-高斯最近交点 + entity-local 存储 + 手算 OpenGL 投影矩阵」的稳定方案。全过程涉及投影数学、PlayCanvas 相机缓存、GSplatData 内部重排、浏览器 API 手势限制等多个深坑。

4.1 功能重构（对应点对齐 v1）

- 交互：选「标记对齐」→ 面板出现；选第 1 个模型=模型 1（亮起），在特征处点左键记入标记组（自动出 ①），完成 3 处；选第 2 个模型=模型 2，按相同顺序点对应特征，满 3 组自动重合、模型对齐完成。
- 状态：`markerModel1 / markerModel2 / markerActive('m1'|'m2') / markerPicking / markerGroups[3]`（每点存 entity-local 坐标）。
- 核心流程：`addMarkerPoint()`（拾取→local）→ `updateMarkerOverlays()`（每帧 local→world→屏幕投影）→ `applyMarkerAlign()`（Kabsch：模型 2 对齐到模型 1）。
- UI：Tab（模型 1/2）+ 3 组对应点圆点（模型 1 青 `#3fd0e0` / 模型 2 粉 `#ef5da8`，实心=已标记）+ 开始/结束标记·应用对齐·清空按钮。

4.2 九轮 Bug 修复全景

Bug 1 · 标记点不显示（03:29 反馈）

现象：模型上点击产生的标记点（屏幕 overlay）不显示。

- 根因 1（投影符号）：`worldToScreen` 中 OpenGL 视图空间前向为 $-Z$ ，原代码 `vz = dx*fx+dy*fy+dz*fz` 未取负 → 前方点的 `vz > -0.01` 为真被 `return null` 隐藏，标记点永远不渲染。
- 根因 2（层级）：`overlay div z-index:9` 低于面板 `.merge-panel` 的 `10`，被面板盖住。

修复

`vz = -(dx*fx+dy*fy+dz*fz)`；`aspect` 改用 `getBoundingClientRect()` CSS 像素宽高比；`overlay z-index` → `20`。

Bug 2 · 应用对齐后模型坍塌（03:29 反馈）

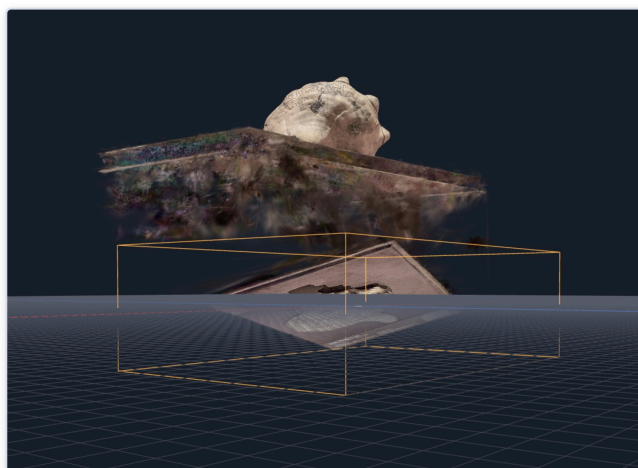


图 4-1 应用标记对齐后其中一个模型坍塌（高斯被压扁/畸变）。

根因： `applyMarkerAlign` 末尾 `setLocalScale(newLocal.getScale())`，把 Kabsch 变换矩阵的尺度/剪切误差写进 `splat` 的 `localScale` → 高斯被压扁；且 3 点近似共线会产生病态矩阵，未校验。

修复

- 三角形面积退化检查： `triangleArea(w1) × triangleArea(w2) < 1e-12` → 报错返回。
- `isRigidTransformValid(T)`：16 项有限 + 行列式 $\in [1e-6, 1e6]$ 。
- 保持 `oldScale`：刚性对齐不改 `scale`（坍塌根因修复）。
- `extractRotationQuat(m)`：正交归一化基提取纯旋转，防 `scale/shear` 污染。

Feature 3 · 撤销 / 重做 (03:29 需求)

- 双栈快照 (`local pos/rot/scale`)，深度 20 (`MAX_HISTORY`)；每次新操作清空 redo。
- 面板「↔ 撤销 / ↶ 重做」按钮 + 快捷键 `Ctrl+Z` / `Ctrl+Y` / `Ctrl+Shift+Z`。
- 坑：类字段类型注解不能用 `typeof this.undoStack`，需显式 `HistoryEntry[]` 接口。

Bug 4 · 标记不跟随模型表面 (04:02 反馈)

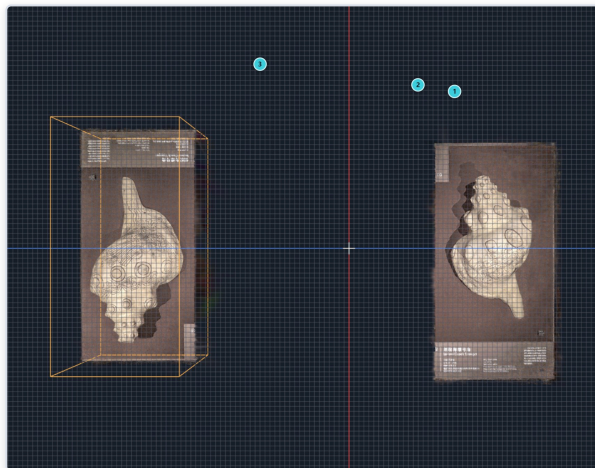


图 4-2 标记点没有紧贴模型表面，移动模型/切换视角时乱窜。

根因： `worldToScreen` / `pickRayManual` 手算 `gluLookAt` 基 (`forward=normalize(target-camPos)`，`side=cross(forward, worldUp=(0,1,0))`)，与 `PlayCanvas` 相机经 `calcCameraUp` 平滑后的实际 `up` 不一致；俯视/仰视时 `side` 退化。

修复

两处改用 `camera.getWorldTransform()` 实际世界基：`right=getX() / camUp=getY() / camBack=getZ()`；视图空间 `vx=d·right`，`vy=d·camUp`，`vz=d·camBack`。

Bug 5 · 标记仍偏移 (04:17 反馈)

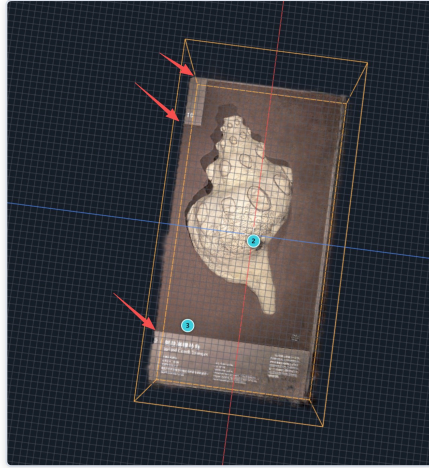


图 4-3 标记点跑到包围盒另一侧；模型 2 第一次点击“没反应”、第二次直接变第 2 个点。

根因： `pickScreen()` 用 `cam._camera?.getInvViewProjMatrix?.()`（缓存矩阵与 GPU 不一致），而 `worldToScreen` 用另一套来源——拾取与投影不同源；且 `aspect` 用了 CSS 而相机投影用 `buffer` ($DPR > 1$ 错位)。模型 2 第 1 个点被投到不可见位置，表现为“没反应”。

修复

`worldToScreen` 改读 `camera.camera.fov / aspectRatio`（公开 getter）+ 实体世界变换；
`pickScreen` 废弃 `_camera` 路径，同一套参数手算射线，与投影严格互逆。

Bug 6 · 绑定高斯 splat (splatIndex 方案) (05:09 反馈)

用户要求：“标记点直接打在被点击位置的高斯点的高斯核上面，直接绑定”。

- `raycastPick` 返回值加 `index`（被命中 splat 在 `getProp('x/y/z')` 中的下标）。
- `markerEls` 存 `splatIndex`；每帧 `getProp('x/y/z')[idx]` × 当前世界变换 → 投影。
- `local` 坐标直接读 `getProp`（比 `inverse(worldTransform)` 精确）。

Bug 7 · 边缘打不上点 (05:09 反馈)

现象：“打点时在边缘打不上，只有点中间，模型居中后才能点击打点”。

根因： `raycastPick` 容差 `fovScale * 0.025` 太紧（约 9–13 像素等效），边缘/稀疏区最近 splat 离射线稍远即被拒。

修复

容差放宽到 `fovScale * 0.08`（≈30+ 像素等效），中心到边缘都能命中。

Bug 8 · 标记跑到画面中心 (05:18 / 05:33 反馈)

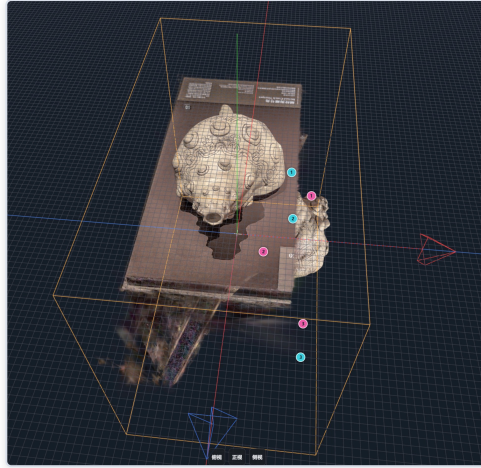


图 4-4 标记点仍“跑得乱七八糟”（splatIndex 方案，标记与点击位置严重错位）。

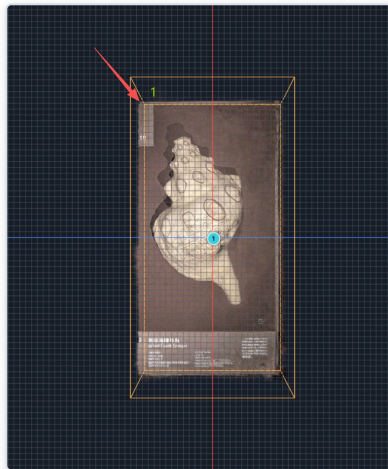


图 4-5 点的是箭头位置（书左上角），标记却出现在雕塑中心——投影矩阵与 GPU 不一致的铁证。

两个关键发现：

- **GSplatData.reorder()**：PLY loader（ply.js:461）会按 Morton 序重排 `property.storage`，splat 下标语义不稳定——splatIndex 方案不可靠。
- **camera.camera.projectionMatrix 是缓存**：公共 getter 内部就是项目记忆警告过的 `_camera` 缓存矩阵，与 GPU 实际渲染不一致 → NDC 整体偏移（标记压向画面中心）。

最终修复（手算 OpenGL 投影矩阵 + 射线-高斯最近交点）

- 新增 `buildManualProj()`：从相机公开的 `fov / aspectRatio / nearClip / farClip` 手算标准 OpenGL 透视矩阵（列主序），与 GPU 用同一组参数构建，像素级一致；不再读任何 `_camera` 缓存。
- `worldToScreen / pickScreen` 都用这份手算矩阵，严格互逆。
- `raycastPick` 的 `bestP` 从 splat 中心改为 `lo + ld * t`（射线到 splat 中心的最近点 = 用户射线真正打到的 3D 位置）。
- `markerEls` 存 `hitLocal: Vec3`（entity-local 命中点），每帧 `entity.transformPoint(hitLocal)` 投影——完全脱离 splat 数据与下标，模型移动/旋转/缩放时标记钉死。

4.3 最终方案定稿（为什么这样一定对）

1. **marker 100% 落在点击像素对应的 3D 位置**（射线-高斯最近交点，不是 splat 中心，不存在 splat 大小导致的视觉偏移）。
2. **不依赖 splat 下标**——绕过 `GSplatData.reorder()` 的 Morton 重排陷阱。
3. **只依赖 entity-local 坐标 × 当前世界变换**，跟随模型，天然支持撤销/重做后的位置恢复。
4. **投影与拾取同源**（手算 OpenGL 矩阵），杜绝缓存矩阵与 GPU 不一致的系统性偏移。

关键代码：手算 OpenGL 透视矩阵（列主序）

```
const fRad = (fov * Math.PI) / 180;
const t = Math.tan(fRad / 2);
return new Mat4().set([
  1 / (aspect * t), 0, 0, 0,
  0, 1 / t, 0, 0,
  0, 0, -(far + near) / (far - near), -1,
  0, 0, -2 * far * near / (far - near), 0
]);
```

5 · 支线探索

5.1 PiP「近小远大」修复 (v9.1, camera-preview.ts)

- **现象**：画中画（PiP）预览里远处物体反而变大。
- **根因**：每个 `GSplatInstance` 只有一个 `orderTexture + sorter`（共享可变状态），PiP 相机滞留污染主排序。
- **修复**：独立 PiP 排序管线（第二个 `GSplatSorter + R32U pipOrder`），swap 隔离；同步 CPU 深度排序（`camDir=getZ()` → `invert(worldTransform).transformVector()` → `dot(center, camDirLocal)` 升序），每 10 帧 1 次，导出期间 `lockedRenderMode` 闸门跳过。

5.2 X/Y/Z 正交正视视角模型消失 (2026-08-04)

- **触发**：正交投影 + 精确轴向 ($\text{azim} \in \{0,90,180,270\}$ 且 $\text{elev}=0$)。
- **根因**：`gsplatCorner.js` 的 `initCornerCov` 在精确轴向正交下 `normalize(vec2(offDiagonal, lambda1 - diagonal1))` 计算出 NaN（屏幕空间协方差退化）→ `corner.offset` 出 clip → 所有 splat 被丢弃。
- **修复**：应用层 shader 检测 NaN/Inf 回退常量偏移；上游 `normalize` 处加 `length > 1e-6` 兜底。
- **经验**：`GSplat` (unified:false 旧管线) `instance.material` 可被 `setDefine+update()` 触发重编译，但 `setParameter` 不生效；验证调试开关优先用 `#define` 或直接硬编码。

5.3 表面平整 L1 / L2 (surface-analyzer / surface-refiner)

- L2 (08-03) : `refineSurfaceLevel2()` — 边缘高斯半径调低 + 游离散点清除; 自适应搜索半径 = 包围盒对角线 $\times$ `radiusFraction` (默认 0.01); 散点评分 = $0.55 \times \text{孤立度} + 0.45 \times \text{透明度} > 0.6$; 异步分块 (8000 点/chunk) 防 UI 冻结。
- L1 性能优化 (08-04) : 仅离群点做邻域 PCA + 上限 64 $\rightarrow$ 100k 点 Phase2 2717ms $\rightarrow$ 75ms。
- L1 四项修复 (08-04) : 排除自身统计 / 阈值 1.5 可调 / 压缩目标压向 `localAvg` / 邻域 PCA 真实法线。
- L1 真分裂 v1-v4 (08-04) : 切平面矩形网格分裂、按原高斯投影椭圆覆盖、仅在物体表面分裂 (`localFlatness < 0.4`)、三类切断 (超大/过扁/夹角超限)。⚠ 因用户要求回退到 11:35 之前, 整体撤销 (当前 L1 = 原始版: 阈值 2.5 硬编码、含自身统计、无 PCA、`edgeSplit` 只计数), 留档供将来重做。

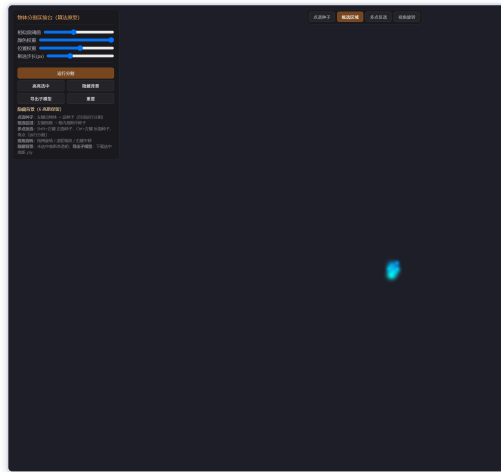


图 5-1 表面平整功能调试截图 (L1/L2 参数面板与效果)。

5.4 平面修整 (Planar Fix, 08-02, 已暂停)

基准面贴平面 + 压平突起高斯; 文件: `planar-fix-box.ts` / `geometry/planar-fix.ts` / `tools/planar-fix-tool.ts` / `ui/planar-fix-panel.ts`; 存档: 平面修整功能存档.md。

5.5 对比工具 (?mode=compare, 08-02 起)

- `startCompareApp()`; 菜单「工具 $\rightarrow$ 高斯训练对比-飞羽实验室」。
- PCA+ICP 对齐、左右分屏、共享相机、24 属性直方图。
- 破洞检测 (v6) : TSPE-GS 透射率 + 累积不透明度 + 48 $\times$ 48 网格 + 像素亮度, `max()` 融合。
- 文件清单: `compare-app/scene/panel/stats/layout/analysis/sync.ts`。

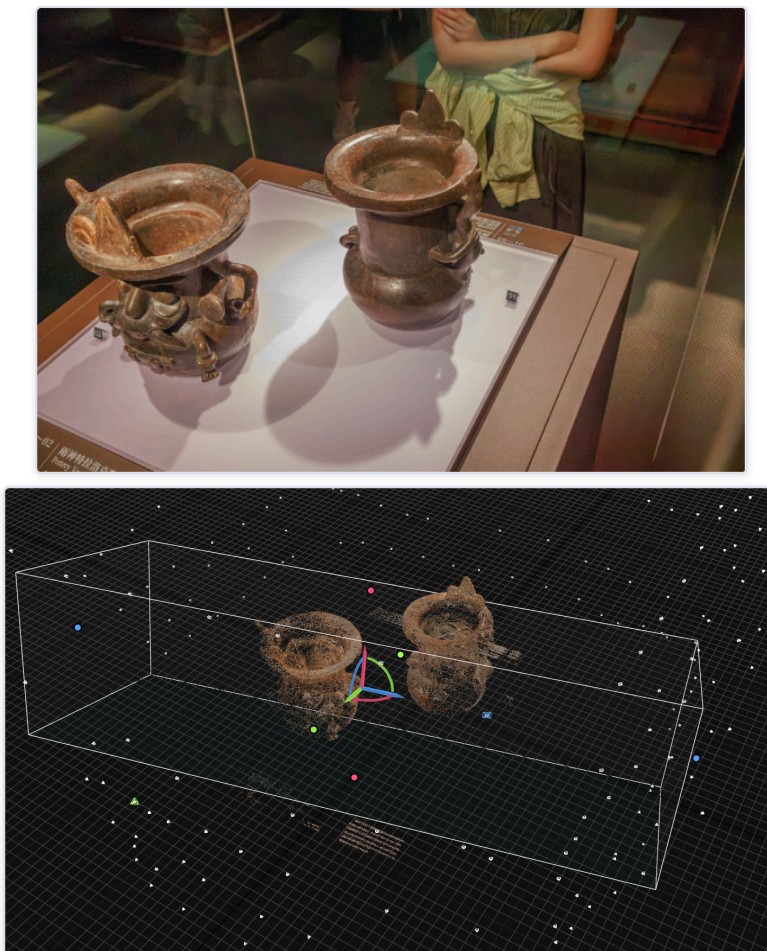


图 5-2 高斯训练对比工具界面（左右分屏、直方图、破洞检测热力）。

5.6 打包发布 (electron-builder)

- portable 目标 → 单文件 `.exe` (免安装)；脚本 `npm run dist:win` = `rollup build` + `electron-builder --win portable`；输出 `release/SplatRoom-${version}.exe`。
- 入口：`electron-main.js` + `electron-preload.js`；版本号改 `package.json` 顶层 `version`。
- safe-delete 假失败**：electron-builder 收尾调 trash 删中间文件，本沙箱的 genie-safe-delete 垫片报错导致退出码 1，但 `building target=portable` 已打印即产物完整，无需重跑。
- 产物校验：python 读首 2 字节应为 `MZ`，再在前部搜 7z 签名 `37 7A BC AF 27 1C`。

5.7 无头浏览器复现 WebGL bug (headless workflow)

- 工具链：`puppeteer-core` + `Edge` + `SwiftShader WebGL` (`--use-gl=angle --use-angle=swiftshader --enable-unsafe-swiftshader --ignore-gpu-blocklist`)。
- 服务器：`npx serve dist -l 3000`；遇 `EPERM` → `Remove-Item dist -Recurse -Force` 再重建。
- 关键判据：**截图文件大小比像素探针可靠** (`drawImage` 抓离屏 canvas 读 `[0,0,0,0]`)；鉴别策略——直接在着色器源码硬编码常量替换正常路径，绕过所有 splat 数据验证绘制调用。
- 大模型 (200k 高斯) `SwiftShader` 单帧约 5s，`page.screenshot()` 必须带 `timeout` + `catch`，否则 `ProtocolError` 卡死。

6 · 关键技术结论

主题	结论
相机投影矩阵	永远不要读 <code>camera.camera.projectionMatrix</code> (内部即 <code>_camera</code> 缓存, 与 GPU 不一致)。手算 OpenGL 标准透视矩阵, 用公开 fov/aspect/near/far, 与 GPU 像素级一致; raycast 用同一份手算矩阵的反投影。
3D 标记绑定	存 射线-高斯最近交点的 <code>entity-local</code> 坐标 (<code>lo + ld * t</code>), 每帧 <code>entity.transformPoint()</code> 投影最稳。splat 下标会被 <code>GSplatData.reorder()</code> 打乱; splat 中心会被 splat 尺寸拖偏; 固定世界点不跟随模型。
GSplatData 内部结构	<code>reorder()</code> (Morton 序) 会重排 <code>property.storage</code> ; <code>calcAabb</code> 对手动构造的数据不可信, 须手动循环算 AABB; <code>getProp('x')</code> 与渲染同源, 但下标语义在 <code>reorder</code> 后变化。
Entity 变换	Entity 无 <code>setLocalTransform</code> , 须分解 <code>setLocalPosition + setLocalRotation + setLocalScale</code> ; 对齐变换保持 <code>oldScale</code> , 用正交化提取纯旋转防 <code>scale/shear</code> 污染。
showSaveFilePicker	必须放在点击 handler 的同步链上; 需要异步准备数据时, 数据准备放进 picker 回调 (picker 先拿 handle, 再调回调取数据)。
GSplat 旧管线调试	<code>instance.material</code> 可被 <code>setDefine+update()</code> 重编译, 但 <code>setParameter</code> 不生效; 调试开关优先 <code>#define</code> 或直接硬编码。
GSplat 排序	多个 <code>GSplatInstance</code> 共享 <code>sorter/orderTexture</code> 会互相污染 → 独立排序管线 + 深度缓冲隔离。

7 · 构建与部署实战经验

坑	现象	解法
safe-delete FAIL_CLOSED	<code>rm -rf dist / fs.unlinkSync / fs.rmSync</code> 被钩子拦截并抛错 (trash aborted), 删除完全不可用	不删除, 用 <code>fs.renameSync</code> 移走旧目录; 残留 <code>dist_bak_*</code> 忽略
EPERM 覆写锁	<code>npm run build</code> 报 EPERM 且失败文件每次不同 (某进程持有 <code>dist deny-write</code> 句柄)	<code>rename dist → dist_bak_<ts></code> , 让 rollup 写全新目录
CloudStudio 部署 400	完整 dist (含 15MB+ .map, 共 30MB) 部署 exec failed (400)	复制 <code>dist_deploy_<ts></code> 剔除 *.map (12MB) 部署成功; 400 偶发可重试同目录
rollup 语法错误难查	构建报错信息不含 "error:" 关键字, grep 会漏掉静默失败	构建验证用 <code>npx tsc --noEmit</code> 或查看完整构建输出

8 · 历次部署链接清单 (08-11 标记对齐日)

阶段	链接 (?mode=merge)	说明
重构完成	2164132c701c442ebd91300ed4fe03be.sh3	标记对齐 3 项修复 (overlay/坍塌/撤销重做)
修复 1	654c525e6b8f4843940fb9e305a64e8d.sh3	投影基改为相机实体世界基
修复 2	b9cf09aa0c9544b28b8171f61dddd5c9.bj9	统一 fov/aspectRatio 来源
修复 3	de6e44a619e34c13a48c511fb756f828.sh3	标记改存世界坐标
绑定 splat	1539fb2a9b01482196f1db234bc5a65d.bj2	splatIndex 方案 (已弃用)
修复 4	6914bb52a8db48c9b4075063da50fda9.gz2	projectionMatrix + 放宽容差 (仍偏)
修复 5	cddb37fea4fc419c85406cdb687663b6.bj5	射线-高斯最近交点 + entity-local
最终	e5582812b8d74930b91bc251fddd456c.sh3	手算 OpenGL 投影矩阵
导出修复	41179f8cd51a475b94ea17911448c3b4.bj4	showSaveFilePicker 手势修复 (最新)

说明：CloudStudio 每次部署会新建 sandbox，链接随之变化；旧链接会失效。最新可用链接以本文档归档时（2026-08-11 13:49）为准。可在「设置 - 数据管理 - 我发布的应用」中管理已发布的应用。

归档说明：本 PDF 由 SplatRoom 项目工作记忆（`~/workbuddy/memory`）与项目 `workbuddy/memory` 整理而成，截图取自开发过程真实反馈。用于项目知识沉淀与后续恢复上下文。